

## **CO4-ASSESSMENT TOOL 1**

**NAME: Nandha Krishna V**

**REG NO:192524012**

**COURSE CODE:CSA0315**

**COURSE:DATA STRUCTURES**

## CO4-ASSESSMENT TOOL 1

1) A system needs to store large data and perform frequent search operations. For this, compare Hashing and Sorting-based search (Binary Search) in terms of:

- a) Time complexity
- b) Space usage
- c) Update operations
- d) Real-world applicability

Conclude which is better for dynamic systems.

When a system stores a large amount of data and performs frequent search operations, Hashing and Sorting-based Binary Search are two commonly used techniques.

### **a) Time Complexity**

Hashing:

- Average search time:  $O(1)$
- Average insertion:  $O(1)$
- Average deletion:  $O(1)$
- Worst-case search:  $O(n)$  due to hash collisions.

Binary Search:

- Data must first be sorted.
- Sorting time:  $O(n \log n)$
- Search time:  $O(\log n)$
- Very efficient when the data is already sorted.

## **b) Space Usage:**

- Hashing requires extra memory for the hash table, generally  $O(n)$ .
- Binary Search requires only  $O(1)$  extra space when performed iteratively.
- However, the data must be maintained in sorted order.

## **c) Update Operations:**

- Hashing supports fast insertion and deletion, generally  $O(1)$  on average.
- In a sorted array, insertion and deletion may require shifting elements, resulting in  $O(n)$  time.
- Therefore, hashing is more suitable when data changes frequently.

## **d) Real-World Applicability:**

Hashing is widely used in:

- Databases and indexing
- Caches
- Dictionaries
- Symbol tables
- Password lookup systems

Binary Search is useful in:

- Searching sorted arrays
- File systems
- Applications where data changes rarely
- Range and ordered-data operations

## **Conclusion:**

For a dynamic system with frequent insertion, deletion, and search operations, Hashing is generally better because it provides  $O(1)$  average search and update operations. Binary Search is better when data is mostly static and maintaining sorted order is important.

2) Compare Quick Sort and Merge Sort based on:

- a) Best, average, and worst-case complexity
- b) Stability
- c) Memory usage
- d) Practical performance

Explain why Quick Sort is often preferred in real applications despite its worst-case complexity.

Quick Sort and Merge Sort are important divide-and-conquer sorting algorithms.

**a) Best, Average and Worst-Case Complexity:**

Case:

- Quick Sort
- Merge Sort
- Best Case
- $O(n \log n)$
- $O(n \log n)$

Average Case:

- $O(n \log n)$
- $O(n \log n)$

Worst Case:

- $O(n^2)$
- $O(n \log n)$

Quick Sort reaches its worst case when the pivot repeatedly produces highly unbalanced partitions.

### **b) Stability:**

Quick Sort: Generally not stable. Equal elements may change their relative order.

Merge Sort: Stable, because equal elements can retain their original order.

### **c) Memory Usage:**

Quick Sort: Usually requires  $O(\log n)$  auxiliary space for recursion when partitions are reasonably balanced. It can work largely in-place.

Merge Sort: Usually requires  $O(n)$  additional memory for arrays because the merging process needs temporary storage.

#### **d) Practical Performance:**

Quick Sort is often very fast in practice because:

- It has good cache locality.
- It performs sorting mostly in-place.
- It has relatively low memory overhead.
- Its average time complexity is  $O(n \log n)$ .
- Merge Sort has predictable  $O(n \log n)$  performance but generally requires more memory.

#### **Why is Quick Sort often preferred?**

Even though Quick Sort has a theoretical worst-case complexity of  $O(n^2)$ , it is often preferred in practical applications because its average performance is  $O(n \log n)$  and it uses less additional memory. Good pivot selection, such as randomized pivot selection or median-of-three, can greatly reduce the possibility of encountering the worst case.

#### **Conclusion:**

Quick Sort is generally preferred when fast average performance and low memory usage are important. Merge Sort is preferred when stability and guaranteed  $O(n \log n)$  performance are required.